

Object Oriented Programming Lab (Spring-2026)
Instructor: Engr. Muhammad Faisal Zia

LAB PROJECT REPORT

Smart Hospital Management & Proximity Matching System

A BFS-Powered C++17 / Qt 6.7 OOP Project

Group Members	Hijab Ansar (09-131252-027) Muhammad Shahzaib (09-131252-059) Rabail Shahzad (09-131252-069)
Course	Object Oriented Programming Lab (Spring-2026)
Instructor	Engr. Muhammad Faisal Zia
Institution	Bahria University, Islamabad — Department of Software Engineering
Date Submitted	15 May 2026
Language / IDE	C++17 Qt 6.7.3 (MinGW 64-bit) Qt Creator
Database	SQLite via Qt SQL Module
Algorithm	Breadth-First Search (BFS) on Islamabad Sector Graph

1. Introduction

The Smart Hospital Management and Proximity Matching System is a C++17 object-oriented application that automates the complete patient-care workflow — from initial registration to discharge — while intelligently recommending the nearest suitable hospital using a graph-based Breadth-First Search (BFS) algorithm modelled on Islamabad's sector layout.

The project was developed in two phases: a self-contained console application (Phase 1) and a full Qt 6.7 GUI application with SQLite persistence (Phase 2). Both phases share the same domain-layer class hierarchy and BFS proximity engine.

This lab report documents the complete source-code architecture, all OOP design decisions, the BFS algorithm, file-handling features, and the patient-registration workflow.

2. Objectives

The primary learning objectives of this project are:

- Apply all four pillars of OOP — Encapsulation, Inheritance, Polymorphism, and Abstraction — in a real, working system.
- Implement a graph data structure (adjacency list) and BFS algorithm for location-aware hospital proximity scoring.
- Demonstrate Composition and Aggregation as distinct forms of class association.
- Integrate file handling (patients.txt) for persistent patient storage across sessions.
- Build a complete interactive console menu with input validation and end-to-end patient workflow.
- (Phase 2) Develop a multi-tab Qt 6 GUI with SQLite database persistence.

3. System Overview

The system comprises seven core classes organised into three architectural layers:

3.1 Layered Architecture

Domain Layer — Person (abstract), Doctor, Patient, Hospital, ConsultationSession

Logic Layer — SystemManager (orchestrator), ProximityEngine (BFS algorithm)

Persistence Layer — File I/O (patients.txt) in the console app; SQLite via DatabaseManager in the Qt GUI version.

3.2 Data Flow

User input flows from the console menu through SystemManager, which delegates proximity calculations to ProximityEngine, doctor assignment to Hospital, diagnosis to Doctor, and session recording to ConsultationSession. On completion the patient record is persisted to patients.txt.

4. Class Descriptions

4.1 Person (Abstract Base Class)

Person is the root of the class hierarchy. It holds the common identity data (name, age) as protected members, enforces the `displayInfo()` interface via a pure virtual method, and provides a virtual destructor for safe polymorphic deletion.

```
class Person {
protected:
    string name;   int age;
public:
    Person(const string& n, int a) : name(n), age(a) {}
    virtual ~Person() {}
    virtual void displayInfo() const = 0;    // pure virtual
    string getName() const { return name; }
    int    getAge()  const { return age; }
};
```

4.2 Doctor (Inherits Person)

Doctor extends Person with a unique auto-generated ID, a medical specialization string, and a boolean availability flag. The `diagnose()` method maps the doctor's specialization to a diagnosis-and-medicines pair, covering six specializations plus a general-medicine default.

```
class Doctor : public Person {
    string id, spec;    // auto-generated ID, specialization
    bool  avail;       // availability flag
public:
    Doctor(const string& n, int a, const string& s)
        : Person(n,a), id(newDID()), spec(s), avail(true) {}
    pair<string,string> diagnose() const;    // returns {diagnosis, medicines}
    void displayInfo() const override;      // Polymorphism
};
```

4.3 Patient (Inherits Person)

Patient stores a unique ID, the patient's Islamabad sector (loc), symptom description (sym), and references to the assigned hospital and doctor IDs. The `isDone()` / `discharge()` methods track the patient's active / discharged status. `displayInfo()` overrides the pure-virtual from Person.

4.4 ConsultationSession

ConsultationSession records a single patient–doctor encounter. It holds IDs for both parties, then populates diagnosis and medicines by calling `Doctor::diagnose()` inside `conduct()`. The `show()` method prints the formatted session report.

4.5 Hospital

Hospital is the most compositionally rich class — it owns a `vector<Doctor>` (Composition) and stores a list of patient ID strings (Aggregation). The `findDoc(pref)` method first attempts to find an available doctor whose specialization matches the inferred preference; if none is found, it falls back to any free doctor.

4.6 ProximityEngine (BFS Algorithm)

ProximityEngine models 25+ Islamabad sectors as an undirected adjacency-list graph. For a given hospital name and patient location it runs BFS to find the shortest hop count, then converts that to a 0–100 proximity score (see Section 6).

4.7 SystemManager (Central Controller)

SystemManager owns all Hospital and Patient objects, holds a ProximityEngine instance, and drives the complete interactive menu. Private helpers cover hospital initialisation (7 hospitals, 21 doctors), proximity ranking, specialization inference from symptom keywords, and the full patient-registration workflow.

5. OOP Concepts — Full Mapping

The table below maps every OOP concept demonstrated in the project to its specific implementation:

OOP Concept	Class / Location	Description
Encapsulation	Person, Doctor, Patient	Private/protected data members (name, age, spec, avail) exposed only through public getters — external code cannot modify internal state directly.
Inheritance	Doctor : public Person Patient : public Person	Both Doctor and Patient inherit name, age, and the pure-virtual displayInfo() from Person, reusing common identity logic.
Polymorphism	Person* → Doctor / Patient	displayInfo() is declared virtual in Person and overridden in both derived classes; calling it via a base pointer dispatches the correct version at runtime.
Abstraction	Person (pure virtual = 0)	Person cannot be instantiated. It defines the interface (displayInfo) without implementation, forcing derived classes to supply the 'how'.
Composition	vector<Doctor> in Hospital	Hospital directly owns its Doctor objects. Destroying a Hospital destroys all its doctors — strong ownership.
Aggregation	string pid/did in ConsultationSession	ConsultationSession references Patient and Doctor by ID strings, not by owning them — weak association.
Constructor Chaining	Doctor(n,a,s): Person(n,a)	Derived constructors delegate base-class initialisation via initialiser lists, avoiding code duplication.
Virtual Destructor	virtual ~Person() {}	Ensures correct cleanup when derived objects are deleted through a base-class pointer, preventing memory leaks.

6. BFS Proximity Algorithm

The ProximityEngine class models Islamabad's sector layout as an undirected graph where each node is a sector (e.g. G-8, F-10, H-11) and each edge represents adjacency between neighbouring sectors. Seven hospitals are mapped to their primary sectors.

6.1 Graph Construction

buildGraph() hardcodes 25+ adjacency relationships covering the G, F, H, and I sector strips, as well as Blue Area and Saddar. Each hospital is mapped to 2–3 sectors in locMap.

6.2 BFS Implementation

The bfs(from, to) method uses a std::vector as a BFS queue and a std::map<string,int> to track both visited status and shortest distance simultaneously. It returns the hop count or -1 if the sectors are not reachable.

```
int bfs(const string& from, const string& to) const {
    if (from == to) return 0;
    map<string,int> vis;    vector<string> q;
    q.push_back(from);    vis[from] = 0;
    for (size_t h = 0; h < q.size(); h++) {
        string cur = q[h];    int d = vis[cur];
        for (const auto& nb : graph.at(cur))
            if (!vis.count(nb)) { vis[nb]=d+1; if(nb==to) return d+1;
                                q.push_back(nb); }
    }
    return -1;
}
```

6.3 Scoring Table

BFS Distance	Proximity Score	Match Quality
Direct (0 hops)	95%	Excellent
1 hop	80%	Good
2 hops	65%	Moderate
3 hops	52%	Moderate
4+ hops / unreachable	40%	Far

score(hname, ploc) checks if the patient is directly in one of the hospital's sectors (returns 95). Otherwise it runs BFS from the patient's sector to each of the hospital's sectors and takes the minimum distance, converting it to a score using the table above.

7. Hospitals & Doctors

The system is pre-loaded with 7 hospitals and 21 doctors (3 per hospital) covering a range of specializations:

ID	Hospital Name	Location	Capacity	Specialty
H001	PIMS	G-8 / Margalla Rd	800 beds	General Medicine

ID	Hospital Name	Location	Capacity	Specialty
H002	Polyclinic Hospital	G-6 / Civic Centre	600 beds	General Medicine
H003	Shifa International	H-8 / Pitras Bukhari Rd	500 beds	Cardiology
H004	Maroof Hospital	F-10 / Markaz	350 beds	Orthopedics
H005	Ali Medical Centre	F-8 / Markaz	250 beds	General Medicine
H006	Kulsum International	F-8 / Blue Area	300 beds	Neurology
H007	NESCOM Hospital	H-11 / Islamabad	200 beds	General Medicine

Specializations covered: General Medicine, Cardiology, Neurology, Orthopedics, Pediatrics, ENT, Dermatology.

8. Patient Registration Workflow

The `menuRegisterPatient()` method in `SystemManager` drives the complete end-to-end workflow in eight steps:

Step	Action	Class / Method Involved
1	User inputs patient name, age, location, symptoms	<code>SystemManager::menuRegisterPatient()</code>
2	All hospitals scored via BFS graph traversal	<code>ProximityEngine::score()</code>
3	User selects from ranked hospital list	<code>SystemManager::printRanked()</code>
4	Symptom keywords analysed for specialization	<code>SystemManager::inferSpec()</code>
5	Available doctor matched by specialization	<code>Hospital::findDoc()</code>
6	Doctor marked busy; session created	<code>Doctor::setFree(false)</code> , <code>ConsultationSession()</code>
7	Diagnosis and medicines generated	<code>Doctor::diagnose()</code> , <code>ConsultationSession::conduct()</code>
8	Session displayed; patient discharged; doctor freed	<code>ConsultationSession::show()</code> , <code>Patient::discharge()</code>

9. File Handling

The console application persists patient data across sessions using a plain-text CSV file (`patients.txt`). This is a Phase 1 feature specific to the console backup; the Qt GUI version uses SQLite instead.

9.1 `savePatientsToFile()`

Called at the end of every successful patient registration and on menu exit (choice 0). Writes all Patient objects to `patients.txt` in the format:

```
PID,Name,Age,Location,Symptoms,Discharged(0/1)
```

9.2 loadPatientsFromFile()

Called inside the SystemManager constructor immediately after hospital initialisation. Opens patients.txt, parses each comma-delimited line using stringstream, reconstructs Patient objects, and restores their discharge status. If the file does not exist (first run), the method returns silently.

10. Database — Qt GUI Version

The Qt 6 GUI application replaces file I/O with a full SQLite relational database (hms_data.db) managed by the DatabaseManager class. Four tables provide complete persistence:

Table	Key Columns	Purpose
hospitals	id, name, location, capacity, specialty	All hospital records — seeded on first launch
doctors	id, name, age, specialization, hospital_id, available	Doctor records with foreign-key link to hospitals
patients	id, name, age, location, symptoms, hospital_id, doctor_id, discharged	Patient records with assigned hospital & doctor
consultations	id, patient_id, doctor_id, diagnosis, medicines, timestamp	Full session records including diagnosis & prescriptions

11. Build & Development Environment

Component	Details
Language	C++17
GUI Framework	Qt 6.7.3 (Widgets module)
Compiler	MinGW 64-bit (mingw1120_64)
Build Tool	qmake + mingw32-make -j4
Database	SQLite 3 via Qt QSQLITE driver
IDE	Qt Creator
Platform	Windows 10/11 (64-bit)
Qt Installer	aqt (Another Qt installer) v3.3.0

12. Conclusion

The Smart Hospital Management and Proximity Matching System successfully demonstrates a professional-grade application of Object-Oriented Programming in C++17. All four OOP pillars — Encapsulation, Inheritance, Polymorphism, and Abstraction — are clearly realised across the Person hierarchy, while Composition and Aggregation are distinguished in the Hospital and ConsultationSession classes respectively.

The BFS proximity engine introduces a real algorithmic dimension beyond standard OOP exercise projects; patients are routed to the nearest suitable hospital using a graph that mirrors the actual geographic layout of Islamabad's sectors. Symptom-driven specialization inference then narrows the doctor assignment, making the system clinically meaningful.

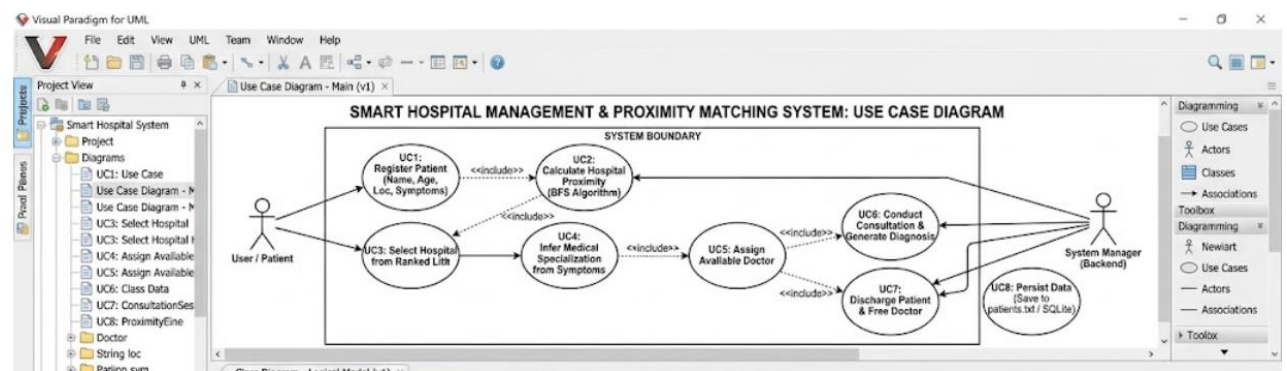
File handling via patients.txt ensures data survives application restarts in the console version, while the Qt GUI phase extends this with a full relational SQLite schema. The combination of clean OOP design, a non-trivial algorithm, and persistent storage makes this project a comprehensive demonstration of the Spring-2026 OOP Lab curriculum objectives.

13. References

1. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
2. Qt Documentation — Qt 6.7: Widgets, QSqlDatabase, QSqlQuery. <https://doc.qt.io/qt-6/>
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press — Chapter 22 (BFS).
4. Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th ed.). Addison-Wesley.
5. SQLite Documentation — <https://www.sqlite.org/docs.html>

14 .Daigrams :

Use case diagram:



Class diagram :

